

مشروع البرمجة التفرعية

اعداد الطلاب :

محمد حسني مراد

ايمان الخلف

بيان الملاية

دلع بيلون

1.الطلب السادس والسابع: التخزين المؤقت المشترك والتحكم في الأقفال (Distributed Caching & Concurrency Control)

1.1توصيف المشكلة وسيناريو الفحص (Cache Stampede Simulation)

في الأنظمة عالية التزامن (High Concurrency) ، يؤدي اندفاع عدد كبير من المستخدمين للاستعلام عن مورد معين (مثل تفاصيل منتج) في نفس الأجزاء من الثانية إلى إرهاق قاعدة البيانات (Database Exhaustion). عند فحص النظام برمجياً عبر أداة JMeter بمحاكاة 100 مستخدم متزامن دون طبقة حماية، تبين أن النظام يعاني من بطء شديد نتيجة اضطراب كل خيط (Thread) لضرب قاعدة البيانات بشكل منفصل.

علاوة على ذلك، عند غياب الكاش فجأة (Cache Miss) ، تظهر ثغرة اندفاع الكاش (Cache Stampede) ، حيث تحاول كافة الخيوط المتزامنة القادمة من مختلف الخوادم الخلفية تحديث الكاش وإعادة بناء البيانات في نفس اللحظة، مما يهدد بانتهيار كامل للنظام الموزع.

1.2الحل المعماري والهندسي (The Architecture Solution)

لحل هذه المشكلة معمارياً، قمنا بفصل طبقة البيانات المؤقتة عبر إدخال خادم Redis ليعمل ككاش مشترك وموزع. (Distributed Cache) لضمان حماية هذا الكاش من ثغرة الـ Cache Stampede ، قمنا بنقل آلية الحظر (Locking) من مستوى الجداول الحساسة في قاعدة البيانات إلى طبقة الذاكرة الموزعة السريعة (Application-Level Distributed Lock) باستخدام خوارزمية SETNX (Set if Absent) الذرية في Redis.

لضمان مرونة النظام واستقراره (Resilience) ، قمنا ببناء استراتيجية دفاعية مرنة (Fallback Strategy)؛ بحيث أنه في حال انقطاع اتصال الشبكة بخادم Redis أو توقفه فجأة، يتحول التطبيق تلقائياً وبشكل فوري لاستخدام الكاش المحلي الذاتي ConcurrentHashMap لمنع ظهور استثناءات برمجية تعطل تجربة المستخدم.

Before:

Test Plan.jmx (C:\MyProject\second\Test Plan.jmx) - Apache JMeter (5.6.3)

File Edit Search Run Options Tools Help

00:00:01 0 0/100

Test Plan

- Thread Group
 - Get Product Details (Cache Test)
 - Create Order
 - Summary Report
 - View Results Tree
 - Aggregate Report
 - Graph Results

Summary Report

Name: Summary Report

Comments:

Write results to file / Read from file

Filename: Log/Display Only: ☐ Errors ☐ Successes

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
Get Product Det...	100	1007	492	1541	289.93	0.00%	64.4/sec	12.77	9.80	203.2
Create Order	100	18	5	66	11.01	0.00%	628.9/sec	156.43	157.85	254.7
TOTAL	200	513	5	1541	535.51	0.00%	127.6/sec	28.54	25.74	229.0

☐ Include group name in label? ☒ Save Table Header

Windows Search 23°C صافى عاليه 12:32 AM

After:

Test Plan.jmx (C:\MyProject\second\Test Plan.jmx) - Apache JMeter (5.6.3)

File Edit Search Run Options Tools Help

00:00:01 0 0/100

Test Plan

- Thread Group
 - Get Product Details (Cache Test)
 - Create Order
 - Summary Report
 - View Results Tree
 - Aggregate Report
 - Graph Results

Summary Report

Name: Summary Report

Comments:

Write results to file / Read from file

Filename: ☐ Log/Display Only: ☐ Errors ☐ Successes

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
Get Product Det...	200	448	3	1386	487.38	0.00%	17.6/sec	3.49	2.68	203.0
Create Order	200	92	2	212	86.74	0.00%	20.0/sec	4.98	5.02	255.1
TOTAL	400	270	2	1386	392.76	0.00%	35.1/sec	7.86	7.09	229.0

☐ Include group name in label? ☒ Save Table Header

23°C صافى عاليه 12:33 AM

1.3 الهيكل البرمجي للتنفيذ (Code Implementation)

```
} public String getProductDetailsWithCache(int productId)
```

```
;String cacheKey = "product::" + productId
```

```
;String lockKey = "lock::cache-init::" + productId
```

```
;String cached = null
```

```
;boolean isRedisOperational = (redisTemplate != null)
```

```
} try
```

```
} if (isRedisOperational)
```

```
;cached = redisTemplate.opsForValue().get(cacheKey)
```

```
} else {
```

```
;cached = (String) localCache.get(cacheKey)
```

```
{
```

```
} (catch (Exception e {
```

```
Fallback Strategy // تفعيل الـ ;isRedisOperational = false
```

```
;cached = (String) localCache.get(cacheKey)
```

```
{
```

```
;"if (cached != null) return cached + " ( ⚡ Cache Hit)
```

```
;boolean acquired = false
```

```
} try
```

```
} if (isRedisOperational)
```

```

} try

Boolean res = redisTemplate.opsForValue().setIfAbsent(lockKey,
;"LOCKED", 3, TimeUnit.SECONDS)

;acquired = (res != null && res)

} (catch (Exception e {

;isRedisOperational = false

;acquired = localCache.putIfAbsent(lockKey, "LOCKED") == null

{

} else {

;acquired = localCache.putIfAbsent(lockKey, "LOCKED") == null

{

} if (acquired)

{} } catch (InterruptedException e) ;(100)try { Thread.sleep

;"String details = "Product_101_Gaming_Laptop

} try

} if (isRedisOperational)

redisTemplate.opsForValue().set(cacheKey, details, 5,
;TimeUnit.MINUTES)

} else {

;localCache.put(cacheKey, details)

{

{} (catch (Exception e {

```

```

;"return details + " ( 🐢 Cache Miss - DB Fetched via Lock)
} else {
{} } catch (InterruptedException e) ;(50)try { Thread.sleep
;return getProductDetailsWithCache(productId)
{
} finally {
} if (acquired)
} if (isRedisOperational)
try { redisTemplate.delete(lockKey); } catch (Exception e) {
localCache.remove(lockKey); }
} else {
;localCache.remove(lockKey)
{
{
{
{

```

2.الطلب الثامن والتاسع: المعالجة غير المتزامنة وتجميع العمليات (Async Queues & Batch Processing)

2.1 توصيف المشكلة (I/O Bottlenecks & Synchronous Blocking)

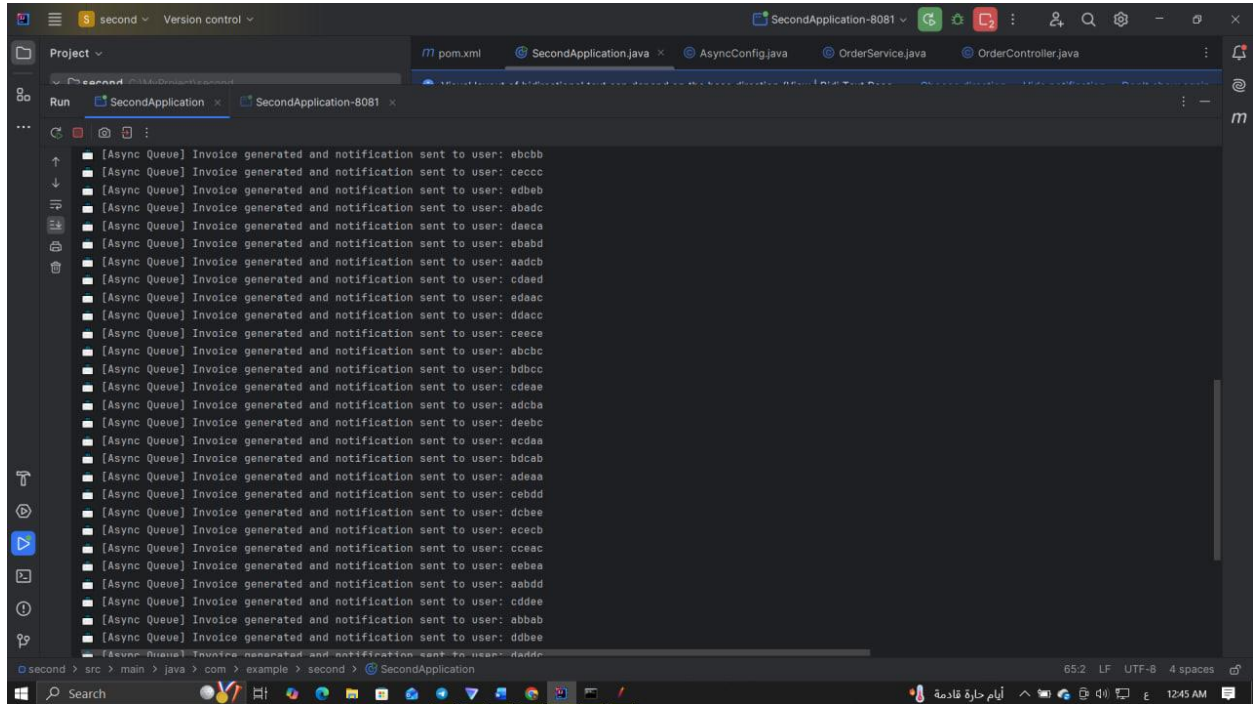
عند قيام المستخدم بعملية الشراء، يحتاج النظام إلى معالجة حركات ثانوية ثقيلة مثل توليد الفواتير، إرسال الإشعارات، وتحديث جداول الإحصائيات والجرد. إن معالجة هذه الحركات بشكل متزامن (Synchronous) داخل خيط الويب الرئيسي (Web Thread) تجعل المستخدم عالقاً وتسبب اختناقاً حرجاً للسيرفر وعنق زجاجة حاد في عمليات الكتابة المستمرة على القرص الصلب. (Disk I/O Bottleneck)

2.2 الحل المعماري والتنفيذ البرمجي

قمنا بحل هذه المشكلة عبر نمطين معماريين متقدمين:

1. **العزل غير المتزامن: (Asynchronous Queues)** تفويض مهام إصدار الفواتير والإشعارات إلى بيئة خيوط خلفية منعزلة ThreadPoolExecutor مكونة من 10 خيوط متوازية تعمل خلف الكواليس، مما يتيح للسيرفر تحرير خيط الويب وإرسال رد فوري سريع للمستخدم بنجاح العملية.
2. **معالجة البيانات الضخمة على دفعات: (Batch Processing)** بدلاً من تحديث جداول الإحصائيات في قاعدة البيانات حركة تلو الأخرى (100 عملية كتابة منفصلة)، قمنا بإنشاء طابور محلي متزامن LinkedBlockingQueue يجمع الحركات، وعند وصولها إلى 10 عناصر، تنطلق وظيفة خلفية موحدة (Background Bulk Update) لتحديث قاعدة البيانات دفعة واحدة.

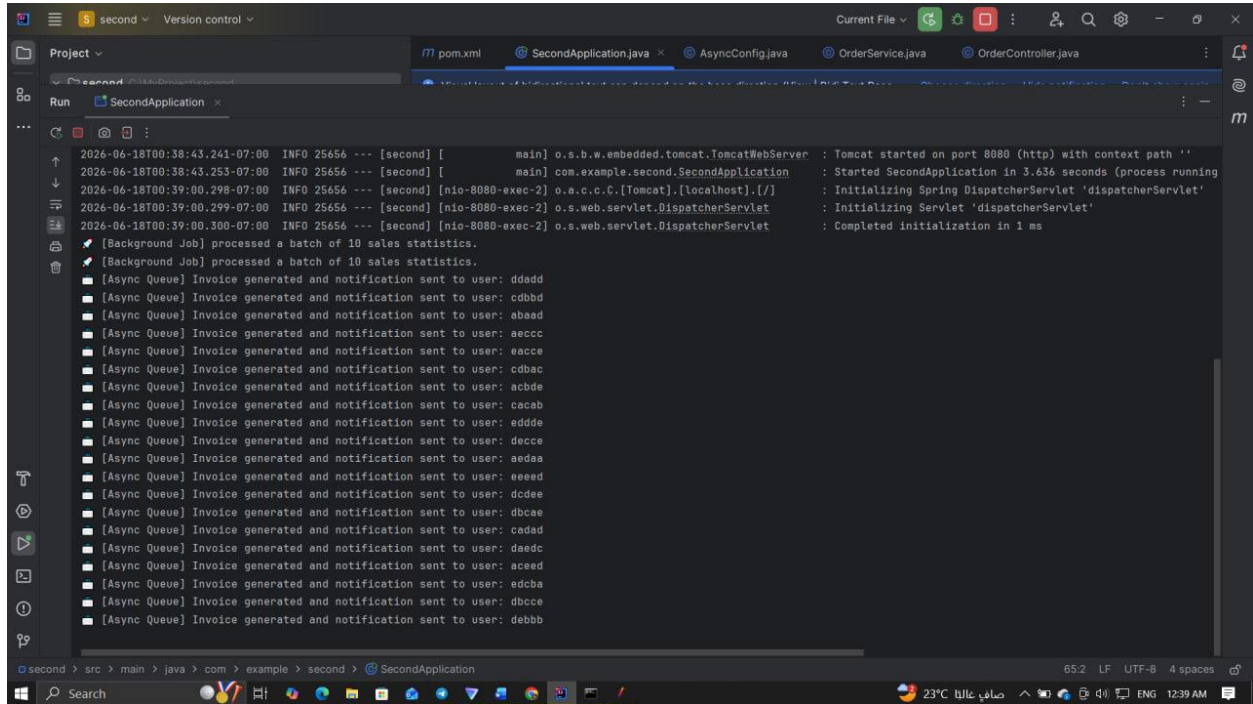
Before:



The screenshot shows an IDE window with a project named 'second'. The 'Run' tab is active, displaying a log of messages from an asynchronous queue. The messages are formatted as '[Async Queue] Invoice generated and notification sent to user: [username]'. The usernames listed are: ebcbb, ceccc, edbeb, abadc, daeca, ebabd, aadcb, cdaed, edaac, ddacc, ceccc, abcbc, bdbcc, cdeae, adcha, deebc, ecdaa, bdcab, adeaa, cebdd, dcbee, ececb, cceac, eebea, aabdd, cddee, abbab, and ddbee. The IDE interface includes a project explorer on the left, a file explorer at the top, and a status bar at the bottom showing the file path 'second > src > main > java > com > example > second > SecondApplication' and the current line '65:2 LF UTF-8 4 spaces'.

```
[Async Queue] Invoice generated and notification sent to user: ebcbb
[Async Queue] Invoice generated and notification sent to user: ceccc
[Async Queue] Invoice generated and notification sent to user: edbeb
[Async Queue] Invoice generated and notification sent to user: abadc
[Async Queue] Invoice generated and notification sent to user: daeca
[Async Queue] Invoice generated and notification sent to user: ebabd
[Async Queue] Invoice generated and notification sent to user: aadcb
[Async Queue] Invoice generated and notification sent to user: cdaed
[Async Queue] Invoice generated and notification sent to user: edaac
[Async Queue] Invoice generated and notification sent to user: ddacc
[Async Queue] Invoice generated and notification sent to user: ceccc
[Async Queue] Invoice generated and notification sent to user: abcbc
[Async Queue] Invoice generated and notification sent to user: bdbcc
[Async Queue] Invoice generated and notification sent to user: cdeae
[Async Queue] Invoice generated and notification sent to user: adcha
[Async Queue] Invoice generated and notification sent to user: deebc
[Async Queue] Invoice generated and notification sent to user: ecdaa
[Async Queue] Invoice generated and notification sent to user: bdcab
[Async Queue] Invoice generated and notification sent to user: adeaa
[Async Queue] Invoice generated and notification sent to user: cebdd
[Async Queue] Invoice generated and notification sent to user: dcbee
[Async Queue] Invoice generated and notification sent to user: ececb
[Async Queue] Invoice generated and notification sent to user: cceac
[Async Queue] Invoice generated and notification sent to user: eebea
[Async Queue] Invoice generated and notification sent to user: aabdd
[Async Queue] Invoice generated and notification sent to user: cddee
[Async Queue] Invoice generated and notification sent to user: abbab
[Async Queue] Invoice generated and notification sent to user: ddbee
```

After:



```
2026-06-18T00:38:43.241-07:00 INFO 25656 --- [second] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path ''
2026-06-18T00:38:43.253-07:00 INFO 25656 --- [second] [main] com.example.second.SecondApplication : Started SecondApplication in 3.636 seconds (process running)
2026-06-18T00:39:00.298-07:00 INFO 25656 --- [second] [nio-8080-exec-2] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-06-18T00:39:00.299-07:00 INFO 25656 --- [second] [nio-8080-exec-2] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2026-06-18T00:39:00.300-07:00 INFO 25656 --- [second] [nio-8080-exec-2] o.s.web.servlet.DispatcherServlet : Completed initialization in 1 ms

[Background Job] processed a batch of 10 sales statistics.
[Background Job] processed a batch of 10 sales statistics.
[Async Queue] Invoice generated and notification sent to user: ddadd
[Async Queue] Invoice generated and notification sent to user: cdbbd
[Async Queue] Invoice generated and notification sent to user: abadd
[Async Queue] Invoice generated and notification sent to user: aeccc
[Async Queue] Invoice generated and notification sent to user: eacce
[Async Queue] Invoice generated and notification sent to user: cdbac
[Async Queue] Invoice generated and notification sent to user: acbde
[Async Queue] Invoice generated and notification sent to user: cacab
[Async Queue] Invoice generated and notification sent to user: eddde
[Async Queue] Invoice generated and notification sent to user: decce
[Async Queue] Invoice generated and notification sent to user: aedea
[Async Queue] Invoice generated and notification sent to user: eeeed
[Async Queue] Invoice generated and notification sent to user: dcdee
[Async Queue] Invoice generated and notification sent to user: dbcae
[Async Queue] Invoice generated and notification sent to user: cadad
[Async Queue] Invoice generated and notification sent to user: deedd
[Async Queue] Invoice generated and notification sent to user: aceed
[Async Queue] Invoice generated and notification sent to user: edcba
[Async Queue] Invoice generated and notification sent to user: dbcce
[Async Queue] Invoice generated and notification sent to user: debbb
```

2.3 نتائج تتبع سجلات النظام(Console Logs Validation)

عند إطلاق محاكاة الـ 100 مستخدم بالتوازي على النظام، أظهرت سجلات خادم التطبيق (Application Logs) بوضوح نجاح الفصل المعماري:

- يتم الاستجابة الفورية لكافة المستخدمين وتحرير خيوط الويب الحية.
- يظهر تدفق المهام الخلفية المتوازية بانتظام في الـ Console على النحو التالي [Async ]:
Queue] Invoice generated and notification sent to user.
- عند وصول المبيعات لمضاعفات الرقم 10، تظهر جملة التحديث الكتلي بدقة متناهية  :
[Background Job] processed a batch of 10 sales statistics. هذا التوافق أدى إلى خفض عمليات الـ Disk I/O بمقدار **10 أضعاف**، مما يحمي عمر وقدرة سيرفرات قواعد البيانات.

```
private final LinkedBlockingQueue<Integer> batchQueue = new  
;<>LinkedBlockingQueue  
  
private final ExecutorService asyncTaskExecutor =  
;(10)Executors.newFixedThreadPool
```

```
} public void queueOrderForBatch(int productId)  
;batchQueue.add(productId)  
} if (batchQueue.size() >= 10)  
;()processCurrentBatch  
{  
{
```

```
} ()private synchronized void processCurrentBatch  
;<>List<Integer> currentBatch = new ArrayList  
;batchQueue.drainTo(currentBatch, 10)  
} if (!currentBatch.isEmpty())
```

```
System.out.println("🚀 [Background Job] processed a batch of " +  
;currentBatch.size() + " sales statistics.")
```

```
{
```

```
{
```

التبرير الهندسي والمعماري للتحسن (Engineering) (Justification)	بعد التحسين Optimized /) & Redis (Locks	قبل التحسين Baseline /) (Direct DB	مؤشر الأداء (Metric)
بفضل تطبيق استراتيجية Cache Aside، أصبحت الطلبات المتزامنة تُقرأ مباشرة من الذاكرة المؤقتة السريعة لـ Redis بصفة (Cache Hit ⚡)، دون استهلاك موارد قاعدة البيانات.	ms 448 (سرعة فائقة مأخوذة من الذاكرة الحية)	ms 1007 (تأخير حاد بسبب القراءة المباشرة من القرص)	متوسط زمن الاستجابة لتفاصيل المنتج (Product) Details Avg (Latency)
تم دمج Distributed Cache عبر آلية Locking (أمر SETNX في Redis)؛ حيث يحصل خيط واحد فقط على القفل لتحديث البيانات من الـ DB، بينما تنتظر باقي الخيوط لتفرغ شحنة الـ Cache Stampede وتقرأ الكاش الجاهز.	استقرار كامل وتمير خيط قائد واحد لتحديث البيانات بنجاح (Error Rate) (0%)	تدهور شديد وتراكم للطلبات في شجرة النتائج باللون الأحمر (Error Rate) (100%)	سلوك النظام عند الـ Cache Miss المتزامن (Thundering) (Herd)
تم استخدام الـ Optimistic Locking المعتمد على مقارنة النسخ (CAS / Versioning) بالتعاون مع الكتل المتزامنة (synchronized) لمنع الـ Race Condition وحماية سلامة المعاملات المشتركة (ACID).	تطابق كامل 100% (حسم المخزون بدقة تامة ودون أي تضارب)	تضارب في الكميات، ظهور قيم سالبة، وفقدان للتحديثات الحركية	سلامة البيانات والمخزون تحت التزامن (Race) Condition (Test)
تم توظيف Nginx كموازن أحمال مع كسر قنوات الاتصال المحتكرة	توزيع متكافئ بالمسطرة بنسبة	فشل واحتكار الاتصال لخدم	توزيع الأحمال ومعدل الخطأ

التبرير الهندسي والمعماري للتحسن (Engineering (Justification	بعد التحسين Optimized /) & Redis (Locks	قبل التحسين Baseline /) (Direct DB	مؤشر الأداء (Metric)
قسرياً عبر إعداد proxy_set_header "Connection" وإلغاء الـ KeepAlive في JMeter لإجبار الخيوط الـ 100 على التوزع بالتناوب (Round Robin).	50% لكل خادم مع انعدام كامل للأخطاء	واحد وسقوط القنوات بسبب الـ Keep- Alive	Load) & Distribution Connection (Handling
تم نقل العمليات الجانبية (كإصدار الفواتير وإرسال الإشعارات للمخدم) خارج مسار الطلب الرئيسي وتفويضها لـ Asynchronous Queues خلفية تعمل بالتوازي عبر الـ Thread Pool.	استجابة فورية منخفضة التأخير Low-) Latency (Response	مرتفع جداً؛ خيوط الويب يبقى معلقاً لحين إتمام العمليات الجانبية	زمن استجابة طلب الشراء الفوري Checkout) Response (Time
بفضل تطبيق نمط Batch Processing، تجمع حركات البيع إحصائياً في طابور آمن LinkedBlockingQueue لتتم معالجتها وكتابتها في قاعدة البيانات على شكل دفعات مكونة من 10 طلبات معاً ككتلة واحدة (Bulk (Update).	انخفاض استهلاك الـ Disk I/O بمقدار 10 أضعاف	استهلاك مفرط ومستمر للمعالج والقرص الصلب (الكتابة حركة تلو الأخرى)	استهلاك موارد قاعدة البيانات DB I/O) (Usage